

Encryption at Rest

How data is stored on disk and how to protect it.

What is stored

Data	Location	Format
Block ledger	STORAGE_PATH/ (RocksDB)	Binary (bincode/JSON in column families)
Raft log	STORAGE_PATH/raft/ (RocksDB)	Protobuf-encoded Raft entries
Chaincode packages	STORAGE_PATH/ (RocksDB CF)	Raw Wasm bytes
Chaincode definitions	STORAGE_PATH/ (RocksDB CF)	JSON-serialized structs
Private data	In-memory (MemoryPrivateDataStore)	Not persisted to disk by default
World state	RocksDB or CouchDB	Key-value pairs with version metadata
TLS certificates	TLS_CERT_PATH, TLS_KEY_PATH	PEM files
Audit log	In-memory (MemoryAuditStore)	Not persisted to disk by default

Current state

RocksDB does not encrypt data at rest by default. Block contents, transaction data, signing keys stored in chaincode definitions, and Raft log entries are written in plaintext to disk.

This is acceptable for development and controlled environments. For production deployments with sensitive data, encryption at rest must be enabled at the filesystem or volume level.

Recommended approach: filesystem-level encryption

The simplest and most portable approach is to encrypt the volume where `STORAGE_PATH` resides.

Linux (LUKS)

```
# Create encrypted partition
cryptsetup luksFormat /dev/sdX
cryptsetup open /dev/sdX rust-bc-data
mkfs.ext4 /dev/mapper/rust-bc-data
mount /dev/mapper/rust-bc-data /data/rocksdb

# Start node with encrypted storage
STORAGE_BACKEND=rocksdb STORAGE_PATH=/data/rocksdb cargo run
```

Docker (encrypted volume)

```
# Create encrypted Docker volume (requires dm-crypt)
docker volume create --driver local \
  --opt type=tmpfs \
  --opt device=tmpfs \
  --opt o=size=10g \
  rust-bc-data

# Or use an encrypted host path
docker run -v /encrypted-mount/rocksdb:/data/rocksdb rust-bc
```

macOS (FileVault)

macOS with FileVault enabled encrypts the entire disk. No additional configuration needed for local development.

Cloud (managed encryption)

Provider	Service	Encryption
AWS	EBS volumes	AES-256 (enabled per volume)
GCP	Persistent Disks	AES-256 (default, automatic)
Azure	Managed Disks	AES-256 (default, automatic)

Key management for disk encryption

- Disk encryption keys should be managed by the infrastructure provider (AWS KMS, GCP KMS, Azure Key Vault)
- The blockchain node does not need to know about disk encryption — it is transparent at the filesystem level
- Rotate disk encryption keys according to your organization's key rotation policy

- TLS private keys (TLS_KEY_PATH) should have restricted file permissions (chmod 600)

What is NOT encrypted

- **Network traffic without TLS:** If TLS is not configured, P2P and API traffic is plaintext. Always set TLS_CERT_PATH and TLS_KEY_PATH in production.
- **In-memory data:** Signing keys, world state cache, and mempool contents exist in plaintext in process memory. The zeroize crate is used to overwrite signing keys on drop.
- **Log output:** Application logs may contain transaction IDs, block heights, and peer addresses. Do not log to unprotected storage.